

Guide de Rapport – Projet Data Science (Format recommandé et Attentes)

Structure recommandée

1. Page de garde

A Inclure : Logo de l'Efrei, titre du projet, noms binôme + promotion / Mastère Data Engineering et IA et Année scolaire, nom du tuteur / tutrice, nom du formateur / formatrice, Date du rapport, mention obligatoire : Projet certifiant RNCP40875 Expert en Ingénierie de Données – Bloc 2 : Pilotage et implémentation de solutions IA

2. Résumé Exécutif (1 page)

Le résumé exécutif doit permettre de comprendre rapidement le projet sans avoir besoin de lire l'ensemble du rapport. Il doit présenter le problème métier traité, le contexte d'application, le dataset utilisé, la tâche prédictive choisie, les principaux choix techniques, les modèles développés, les résultats obtenus et l'outil final proposé. Il faut également mentionner le dashboard développé, l'éventuelle API, les technologies principales utilisées, ainsi que la valeur ajoutée de la solution. Ce résumé ne doit pas être une introduction longue, mais une synthèse claire du travail réalisé, mettant en évidence la démarche Data Science complète : analyse des données, préparation, modélisation, comparaison, interprétation et mise à disposition des résultats dans un outil décisionnel.

3. Introduction et Contexte

Cette partie doit présenter la problématique générale du projet et expliquer pourquoi elle est pertinente dans un contexte professionnel. Selon le sujet choisi, il faudra introduire le contexte métier correspondant : maintenance prédictive industrielle, rétention client et risque de revenus, optimisation du retour sur investissement marketing, ou autre sujet libre validé. L'objectif est de montrer que le projet ne consiste pas uniquement à entraîner un modèle, mais à répondre à un besoin métier réel par une démarche Data Science structurée. Le Machine Learning supervisé doit être présenté comme une approche permettant d'apprendre à partir de données d'entrée associées à une variable cible, afin de produire des prédictions sur de nouvelles situations. Dans le cas de la maintenance prédictive, il peut s'agir d'anticiper une panne, d'identifier un type de défaillance ou d'estimer une durée de vie restante. Dans le cas de la rétention client, il peut s'agir d'identifier les clients à risque ou d'évaluer un revenu à risque. Dans le cas du ROI marketing, il peut s'agir de prédire les ventes ou d'estimer l'impact d'une allocation budgétaire. Les sujets Data Science précisent que le projet doit permettre de passer de données brutes à un outil d'aide à la décision, en intégrant préparation des données, modélisation multi-algorithmes, évaluation comparative, interprétabilité, dashboard et éventuellement API.

4. Analyse du besoin utilisateur

Cette section doit démontrer que vous connaissez l'utilisateur final de votre solution. Il ne suffit pas de dire que le projet utilise un modèle de Machine Learning ; il faut expliquer à qui ce modèle est destiné, quelles décisions il doit soutenir et quelles informations sont utiles pour l'utilisateur métier. Pour un projet de maintenance prédictive, l'utilisateur peut être un responsable maintenance, un ingénieur industriel ou un manager opérationnel qui souhaite prioriser les

interventions et éviter les arrêts non planifiés. Pour un projet de rétention client, l'utilisateur peut être un responsable CRM, marketing ou financier cherchant à prioriser les clients à contacter et à réduire le churn. Pour un projet de ROI marketing, l'utilisateur peut être un CMO, un responsable acquisition ou une direction commerciale souhaitant simuler des budgets et maximiser les ventes. Cette partie doit inclure les objectifs utilisateurs, les scénarios d'usage, les hypothèses, les contraintes, les décisions à soutenir et les indicateurs attendus dans le dashboard.

5. Méthodologie de travail et gestion de projet

Cette section doit présenter la manière dont le projet a été organisé. Il est attendu de décrire la méthodologie adoptée, par exemple Agile, Kanban ou hybride, ainsi que la répartition des tâches entre les membres du binôme. Il faut expliquer les grandes étapes du projet : compréhension du sujet, exploration du dataset, préparation des données, choix de la cible, entraînement des modèles, comparaison des performances, interprétation, développement du dashboard et préparation de la soutenance. Il est recommandé d'intégrer un planning, un mini-Gantt, une roadmap ou une capture d'un tableau de suivi si vous avez utilisé Trello, Notion, GitHub Projects ou un outil équivalent. Cette partie doit aussi présenter les risques rencontrés, tels que la qualité des données, le déséquilibre des classes, les difficultés d'interprétation, les problèmes de performance, les erreurs de déploiement Streamlit ou les limites de temps, ainsi que les solutions mises en œuvre pour les gérer...

6. Référentiel de données

Cette partie doit décrire de manière structurée le dataset utilisé. Il faut présenter la source des données, leur volume, les variables disponibles, les types de données, les variables explicatives, la variable cible choisie, ainsi que la pertinence métier de cette cible. Il ne s'agit pas seulement de copier la sortie de `df.info()` ou `df.describe()`, mais d'expliquer ce que les données représentent dans le contexte du projet. Si le sujet porte sur la maintenance prédictive, les variables peuvent représenter des mesures issues de capteurs industriels, telles que la température, les vibrations, la pression ou le régime moteur. Si le sujet porte sur la rétention client, les variables peuvent représenter des comportements clients, des interactions avec le service, des informations de satisfaction ou des données contractuelles. Si le sujet porte sur le ROI marketing, les variables peuvent représenter des budgets publicitaires, des canaux marketing, des ventes ou des indicateurs de performance. Cette partie doit également expliquer les éventuelles limites du dataset, comme les valeurs manquantes, les variables peu explicites, les outliers, les classes déséquilibrées ou le volume limité.

7. Analyse exploratoire des données et visualisation

L'analyse exploratoire des données doit être une partie importante du rapport. Elle doit montrer que vous avez pris le temps de comprendre les données avant de lancer les modèles. Il faut présenter les distributions principales, les statistiques descriptives, la présence de valeurs manquantes, les valeurs extrêmes, les corrélations, les relations entre les variables explicatives et la cible, ainsi que les premières hypothèses métier. Cette analyse doit s'appuyer sur des visualisations pertinentes, telles que des histogrammes, boxplots, graphiques de corrélation, nuages de points, heatmaps ou graphiques de répartition de la cible.

Pour une tâche de classification, il faut visualiser la distribution des classes et discuter l'éventuel déséquilibre. Pour une tâche de régression, il faut analyser visuellement la distribution de la variable cible, sa dispersion, ses valeurs extrêmes et ses relations avec les variables explicatives. Les visualisations ne doivent pas être uniquement décoratives : chaque graphique doit être

accompagné d'une lecture et d'une conclusion. Les consignes du projet insistent sur le fait qu'il faut distinguer les visualisations d'analyse scientifique, destinées au rapport technique, et les visualisations du dashboard final, qui doivent être orientées utilisateur métier.

8. Préparation, transformation des données et visualisation du prétraitement

Cette section doit expliquer toutes les opérations de préparation réalisées avant la modélisation. Il faut présenter le nettoyage des données, le traitement des valeurs manquantes, la gestion des doublons, le traitement éventuel des outliers, l'encodage des variables catégorielles, la normalisation ou standardisation lorsque cela est nécessaire, ainsi que la création éventuelle de nouvelles variables. Ces choix doivent être justifiés par des observations issues de l'EDA et, lorsque c'est pertinent, accompagnés de visualisations avant/après prétraitement.

Les visuels peuvent permettre de montrer l'effet du nettoyage, de l'imputation, de la normalisation, de la gestion des outliers ou du rééquilibrage des classes. Il faut également préciser comment le dataset a été divisé entre entraînement et test, et si une validation croisée a été utilisée. Les choix doivent être justifiés en fonction des modèles utilisés : par exemple, la standardisation est importante pour certains modèles comme la régression logistique, le SVM ou les réseaux de neurones, tandis qu'elle est moins indispensable pour des modèles d'arbres comme Random Forest. Si le projet contient une classification avec classes déséquilibrées, il faut expliquer les stratégies utilisées, comme le stratified split, les poids de classes, l'ajustement du seuil de décision ou éventuellement des techniques de rééchantillonnage lorsque cela est pertinent.

9. Pipeline IA et architecture

Un schéma explicatif, lisible et obligatoire doit présenter la chaîne de traitement de bout en bout. Ce pipeline doit représenter le passage des données brutes vers une solution exploitable. Il doit inclure le chargement des données, l'analyse exploratoire, le nettoyage, la transformation des variables, la séparation train/test, l'entraînement de plusieurs modèles, l'évaluation comparative, le choix du modèle final, l'interprétation des résultats, la sérialisation éventuelle du modèle, l'intégration dans le dashboard et, si elle est réalisée, l'exposition via une API REST. Le schéma doit refléter votre implémentation réelle. Il peut prendre la forme d'un diagramme simple montrant les composants principaux et leurs interactions. L'objectif est de montrer que votre solution n'est pas un modèle isolé, mais un pipeline Data Science complet, cohérent et reproductible.

10. Implémentation technique

Cette partie doit décrire en détail la solution développée. Il faut présenter l'architecture globale du projet, les langages et librairies utilisés, les choix techniques, l'organisation du dépôt Git, les fichiers principaux, les dépendances, ainsi que les éventuels modules de preprocessing, d'entraînement, d'évaluation, d'inférence et de dashboard. Il faut expliquer les modèles testés, leur rôle, leurs paramètres principaux et les raisons de leur sélection. Le projet doit intégrer plusieurs modèles de Machine Learning, ainsi qu'au moins un modèle de Deep Learning, généralement un MLP pour une tâche de classification ou de régression. L'objectif n'est pas seulement d'obtenir un score élevé, mais de comparer les approches, de comprendre leurs forces et leurs limites, et de justifier le choix final. Les consignes rappellent que le Deep Learning ne doit pas être intégré de manière automatique ou dogmatique : il faut analyser son intérêt réel, son risque d'overfitting, son coût computationnel et sa valeur ajoutée par rapport à des modèles plus simples.

11. Évaluation comparative des modèles

Cette section doit présenter les résultats obtenus par les différents modèles. Pour une tâche de classification, les métriques peuvent inclure l'accuracy, la précision, le recall, le F1-score, la matrice de confusion, le ROC-AUC ou le PR-AUC, selon le contexte. Pour une tâche de régression, les métriques peuvent inclure la MAE, la RMSE, le R^2 et éventuellement l'analyse des résidus. Il faut produire un tableau comparatif clair et interprété. La comparaison ne doit pas se limiter à annoncer le meilleur score. Il faut expliquer pourquoi un modèle est retenu : performance, stabilité, généralisation, interprétabilité, complexité, temps d'entraînement, facilité de déploiement et cohérence avec le besoin métier. Les compétences évaluées demandent explicitement de développer plusieurs modèles, d'utiliser des métriques adaptées, de comparer les résultats et de justifier le modèle final en lien avec les attentes métier.

12. Interprétabilité et analyse métier des résultats

Cette partie doit montrer que vous êtes capables d'expliquer les résultats du modèle et de les relier au contexte métier. Il faut présenter les variables les plus importantes, les coefficients si vous utilisez un modèle linéaire, les importances de variables pour des modèles d'arbres, ou des méthodes comme SHAP si elles sont mises en œuvre. L'objectif est d'expliquer pourquoi le modèle prend certaines décisions et quelles variables influencent le plus la prédiction. Cette interprétation doit être traduite dans un langage compréhensible pour un utilisateur non spécialiste. Par exemple, dans un projet de maintenance, il faut expliquer comment certaines plages de température, vibration ou pression peuvent être associées à un risque plus élevé. Dans un projet de churn, il faut expliquer quels comportements clients sont liés au risque de départ. Dans un projet de ROI marketing, il faut expliquer quels budgets ou canaux semblent avoir le plus d'impact sur les ventes. Cette partie est importante car un modèle performant mais incompréhensible a une valeur limitée dans un contexte professionnel.

13. Interface utilisateur et prototype

Cette section doit présenter le dashboard développé avec Streamlit, Dash ou un outil équivalent. Le dashboard ne doit pas être une simple reproduction des graphiques de l'EDA. Il doit être pensé comme une interface décisionnelle destinée à l'utilisateur métier. Il doit permettre de visualiser les indicateurs clés, de saisir ou modifier un scénario, d'obtenir une prédiction en temps réel, de comprendre le résultat et d'accéder à une explication simple des variables influentes. Des captures d'écran doivent être intégrées au rapport pour montrer les principales pages du prototype, les formulaires, les graphiques, les KPI, les résultats de prédiction et les messages d'interprétation. Le projet demande explicitement de transformer un modèle académique en outil opérationnel ou décisionnel, ce qui rend cette section centrale dans l'évaluation.

14. API REST (partie optionnelle)

Si une API a été développée, cette partie doit expliquer son rôle et son fonctionnement. Il faut présenter la technologie utilisée, par exemple FastAPI ou Flask, les endpoints disponibles, le format des données en entrée, le format de la réponse, la gestion des erreurs et la manière dont l'API peut être reliée au dashboard ou à une application externe. L'API peut inclure un endpoint /predict, un endpoint /health et éventuellement un endpoint /model-info. Cette partie est optionnelle mais valorisante, car elle introduit la logique d'industrialisation d'un modèle. Elle permet de montrer que le modèle n'est pas seulement utilisé dans un notebook, mais peut être exposé comme un service exploitable par d'autres systèmes. Les consignes du projet rappellent

que l'industrialisation par API est une extension permettant de rapprocher la solution des pratiques professionnelles.

15. Résultats et tests de démonstration

Cette section doit présenter les tests réalisés pour vérifier le bon fonctionnement de la solution. Il faut montrer plusieurs scénarios d'utilisation, par exemple un cas à faible risque et un cas à risque élevé, un client peu susceptible de churner et un client à haut risque, ou encore une configuration budgétaire marketing faible et une configuration plus performante. Les résultats doivent être présentés de manière claire, avec des captures d'écran du dashboard ou des exemples d'entrées et sorties. Il faut également discuter le comportement du modèle, les erreurs éventuelles, les cas limites et la cohérence des prédictions. Cette partie doit montrer que la solution fonctionne concrètement et qu'elle peut être utilisée pour soutenir une décision métier.

16. Gouvernance, responsabilité et limites

Cette section doit prendre du recul sur la solution développée. Il faut discuter la qualité des données, la traçabilité du pipeline, les limites du dataset, les risques de biais, le risque de surapprentissage, la robustesse du modèle, la dérive possible des données dans le temps, les enjeux de confidentialité, ainsi que les limites de l'interprétabilité. Si la solution devait être déployée dans un contexte réel, il faudrait prévoir un suivi de performance, une supervision des prédictions, une mise à jour régulière du modèle, une validation humaine dans les cas critiques et une documentation technique claire. Cette partie permet de montrer que vous ne considérez pas le modèle comme une réponse absolue, mais comme un outil d'aide à la décision qui doit être évalué, surveillé et amélioré.

17. Limites et pistes d'amélioration

Cette partie doit présenter une analyse critique honnête du projet. Il faut expliquer ce qui a bien fonctionné, ce qui a été plus difficile et ce qui pourrait être amélioré. Les limites peuvent concerner la taille du dataset, la qualité des variables, le déséquilibre des classes, le manque de données temporelles, la difficulté à interpréter certains modèles, la performance limitée du Deep Learning, l'absence de déploiement cloud ou l'absence de monitoring. Les pistes d'amélioration peuvent inclure l'enrichissement des données, l'ajout de nouvelles variables, l'optimisation des hyperparamètres, l'utilisation de SHAP, l'ajout d'une base de données, l'amélioration du dashboard, l'intégration d'une API, le déploiement cloud ou l'ajout d'un système de suivi de dérive. Cette section doit montrer votre capacité d'analyse critique et votre maturité professionnelle.

18. Conclusion

La conclusion doit synthétiser le travail réalisé et rappeler la valeur ajoutée du projet. Elle doit expliquer comment vous êtes passés d'un dataset brut à une solution Data Science complète, intégrant préparation des données, modélisation, évaluation, interprétabilité et interface utilisateur. Elle doit aussi rappeler le modèle retenu, les résultats principaux, l'intérêt métier de la solution et les compétences développées. La conclusion doit insister sur le fait que l'objectif du projet n'est pas uniquement d'obtenir le meilleur score, mais de construire une solution robuste, explicable, défendable et exploitable dans un contexte réel. Les recommandations du sujet précisent d'ailleurs que la réussite repose sur une démarche méthodologique rigoureuse, structurée et défendable, capable de justifier les choix comme dans un contexte professionnel.

19. Annexes optionnelles

Les annexes peuvent contenir les éléments complémentaires qui alourdiraient le corps du rapport. Il est possible d'y intégrer l'arborescence du projet, des extraits de code, le lien GitHub, le fichier requirements.txt, des captures supplémentaires, des résultats détaillés, des exemples de requêtes API, des schémas techniques, des tableaux de métriques complets ou toute documentation utile. Les annexes ne remplacent pas l'analyse principale : elles doivent compléter le rapport sans disperser les éléments essentiels.

Consignes générales de rédaction

- 20–30 pages (hors annexes)
- Police simple (Times New Roman 12, Arial/Calibri 11), interligne 1,
- Style professionnel et structuré
- Important : Illustrer avec captures d'écran, schémas, tableaux

Consignes de Soutenance (Format recommandé et Attentes)

La soutenance doit être **courte, professionnelle et structurée**.

Durée totale : **9 à 10 minutes maximum**, intégrant la session de Q/R.

Conseil : Pendant la présentation, allez directement à l'essentiel et évitez d'entrer trop dans les détails.

Contenu recommandé pour les diapos de soutenance

Les diapos doivent reprendre la structure du rapport, en version synthétique et centrée sur les éléments essentiels.

Elles peuvent être réorganisées selon votre préférence, à condition de conserver un fil conducteur clair, une cohérence logique et un contenu adapté au temps de présentation.

Présentation du projet et contexte

- Problématique adressée
- Contexte métier et utilisateurs visés
- Objectif du projet
- Type de tâche : classification, régression ou les deux
- Pitch en une phrase, par exemple : “Un système intelligent multi-modèles permettant de prédire le risque de churn et d’aider les équipes marketing à prioriser les actions de rétention.”

Analyse du besoin utilisateur

- Personne cible : responsable métier, manager, analyste, équipe opérationnelle
- Scénarios d’usage : prédiction, priorisation, simulation, aide à la décision
- Contraintes et hypothèses métier
- Indicateurs utiles pour l’utilisateur final
- Valeur ajoutée attendue de la solution

Méthodologie de travail et organisation

- Approche de gestion de projet : Agile, Kanban ou hybride
- Planification : jalons, Gantt ou mini-roadmap
- Répartition des tâches et collaboration en binôme
- Outils utilisés : GitHub, Trello/Notion, Teams, Google Drive, etc.
- Gestion des risques et imprévus : qualité des données, retard, erreurs techniques, overfitting

Référentiel de données et dataset

- Source des données utilisées
- Description rapide du dataset
- Nombre de lignes, colonnes et types de variables
- Variable cible retenue

- Variables explicatives principales
- Format utilisé : CSV, JSON, SQL ou autre
- Extraits illustratifs si nécessaire

Analyse exploratoire des données et visualisation

- Statistiques descriptives principales
- Distribution des variables importantes
- Distribution de la variable cible
- Valeurs manquantes, doublons, outliers
- Corrélations et relations variables/cible
- Visualisations clés : histogrammes, boxplots, heatmap, bar charts, scatter plots
- Premières hypothèses métier issues de l'EDA

Préparation et transformation des données

- Nettoyage des données
- Traitement des valeurs manquantes
- Gestion des outliers
- Encodage des variables catégorielles
- Normalisation ou standardisation si nécessaire
- Création éventuelle de nouvelles variables
- Split train/test et validation croisée éventuelle
- Gestion du déséquilibre des classes si applicable
- Visualisation avant/après prétraitement si pertinente

Architecture globale et choix technologiques

- Vue d'ensemble de la solution : frontend / dashboard, backend, , API
- Rôle de chaque composant
- Technologies utilisées : Python, Pandas, Scikit-learn, TensorFlow/Keras ou PyTorch, Streamlit/Dash, ou autres
- Justification synthétique des choix techniques
- Organisation du dépôt Git
- Principales librairies utilisées

Pipeline Data Science et architecture IA

- Schéma visuel clair du pipeline de bout en bout
- Chargement des données
- Préprocessing
- Feature engineering
- Entraînement des modèles ML/DL
- Évaluation comparative
- Sélection du modèle final
- Interprétabilité
- Intégration dashboard
- API REST (si implémentée)
- Mini-focus sur les étapes critiques : preprocessing, choix des métriques, comparaison, interprétation

Implémentation technique (éléments clés)

- Modèles Machine Learning testés
- Modèle Deep Learning retenu ou testé
- Raisons du choix des modèles
- Hyperparamètres principaux
- Métriques utilisées selon la tâche
- Comparaison des performances
- Méthode d'interprétabilité : coefficients, feature importance, SHAP ou autre
- Sérialisation du modèle si réalisée
- Gouvernance : qualité, traçabilité, reproductibilité, limites et responsabilité

Résultats et tests

- Tableau comparatif des modèles
- Métriques principales obtenues
- Modèle final retenu
- Justification du choix final
- Comportement observé du modèle
- Vérification du bon fonctionnement
- Tests sur plusieurs exemples ou scénarios
- Interprétation rapide des résultats

Démonstration live

- Présentation rapide du dashboard
- Input utilisateur ou scénario de simulation
- Affichage des KPI ou graphiques principaux
- Prédiction en temps réel
- Interprétation du résultat
- Visualisation des variables importantes
- Exemple de recommandation ou décision métier

Attention :

- Préparer en amont les entrées pour éviter de perdre du temps
- Prévoir un cas alternatif si le temps le permet
- Avoir des screenshots en backup et/ou vidéo enregistrée si la démo live ne fonctionne pas

Dashboard décisionnel

- Objectif du dashboard
- Utilisateur métier visé
- KPI affichés
- Graphiques et filtres interactifs
- Fonction de prédiction ou simulation
- Résultat affiché de manière compréhensible
- Aide à la décision apportée par l'interface
- Différence entre visualisations EDA et visualisations métier

API REST (partie optionnelle)

- Rôle de l'API dans la solution

- Technologie utilisée : FastAPI, Flask ou autre
- Endpoint /predict, Endpoint /health, Endpoint /model-info si disponible
- Exemple simple d'entrée/sortie JSON, etc.
- Gestion des erreurs
- Lien éventuel avec le dashboard

Limites et pistes d'amélioration (important)

- Limites du dataset
- Qualité ou volume des données
- Déséquilibre éventuel des classes
- Risque d'overfitting ou underfitting
- Limites du modèle Deep Learning
- Limites de l'interprétabilité
- Limites du dashboard ou de l'API
- Améliorations envisagées : tuning, SHAP, nouvelles données, base de données, déploiement cloud, monitoring, etc.

Conclusion

- Ce que vous avez réalisé, et ce que vous avez appris
- Modèle final et résultats principaux
- Valeur métier démontrée
- Compétences visées et acquises : EDA, preprocessing, Machine Learning, Deep Learning, évaluation, interprétabilité, dashboard, API optionnelle, gestion de projet
- Ouverture vers une industrialisation ou amélioration future

Consignes générales pour la soutenance

- Cette structuration est proposée à titre d'inspiration.
- Vous pouvez adapter l'ordre des slides et organiser votre présentation comme vous le jugez le plus pertinent, **à condition de respecter le temps imparti et de garder un fil conducteur clair.**
- Il n'est pas nécessaire de faire une slide pour chaque point. **Vous pouvez regrouper intelligemment plusieurs éléments dans une même diapositive afin de gagner du temps et d'éviter une présentation trop chargée.**
- Au début, **pensez à vulgariser le contexte** : expliquez simplement la problématique, l'utilisateur visé, le besoin métier et la valeur ajoutée de votre solution Data Science.
- Lors de la démonstration, mettez en relief les points techniques importants réalisés : preprocessing, modèles testés, comparaison des performances, interprétabilité, dashboard, prédiction en temps réel ou API si elle existe.
- La présentation doit rester claire, logique et professionnelle. Évitez de trop détailler le code.
- **Privilégiez l'explication des choix, des résultats, des limites et des améliorations possibles.**

Attention: Préparer en amont votre démo et les entrées pour éviter de perdre du temps)

- Prévoir au moins 1 cas alternatif si le temps le permet
- Avoir screenshots en backup si API lente ou courte démo enregistrée